MINOR PROJECT – 23BCP142 & 23BCP516D

CodeSync

Project-Aware AI for Safe Code Refinement

An AI tool that improves code one piece at a time, and automatically checks that the rest of the project still works before it accepts any change.

The problem we are solving

AI coding tools today look at one file at a time. You give them a file, they give you a cleaner version back. The problem is that the file they were given is usually connected to many other files, and the AI never sees those connections.

So the AI can produce a version of the file that is genuinely better on its own, and still break the project. Here is the kind of thing that happens:

```
# before
def parse_row(line, sep=",", strict=False):

# after - the AI's improved version def parse_row(line, *, delimiter=","):
```

The new version is tidier and clearer. But three other files in the project still call this function the old way, and all three now crash. Nothing inside the file is wrong. The damage is entirely at the joins between files.

A human reviewer would catch this, because a human knows the rest of the project. Our tool gives the AI that same awareness, and then verifies the result instead of trusting it.

The key idea

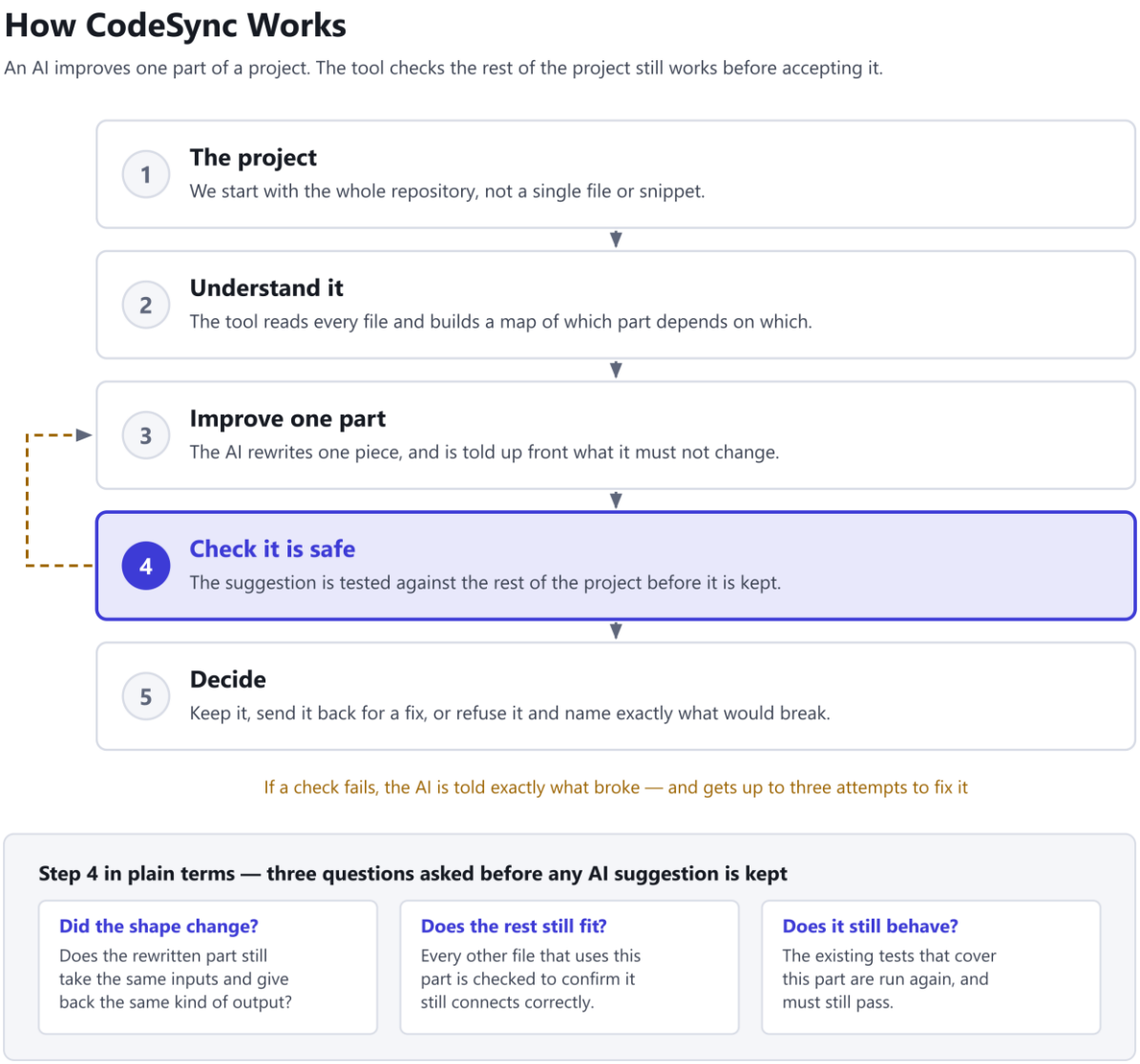
We take a snapshot of what each piece of code promises - what it takes in and what it gives back - before the AI touches it. After the AI rewrites it, we take the snapshot again and compare. If that promise changed in a way that breaks anything else in the project, the change does not get in.

Why this is worth building

- **It is a real gap.** Existing AI review tools suggest improvements but cannot tell you whether applying them is safe.
- **It is measurable.** We can count how many breaking changes the tool catches, which makes it a proper experiment and not just a demo.
- **It is honest about limits.** When the tool is not sure a change is safe, it refuses it and explains why. A refusal is a useful answer.

How it works

Five steps. The fourth one is the part that makes this project different from an ordinary AI code reviewer.



Three things worth noting about the checking step

- **Nothing touches the real project.** The AI's version is written into a temporary copy. If it fails, the copy is thrown away and the original was never altered.
- **Cheap checks run first.** A broken suggestion is caught in milliseconds and never reaches the slow checks. Only suggestions that already look sound get tested properly.
- **A refusal is a result.** If the tool cannot confirm a change is safe, it says no and lists the files that would have broken. That is the tool doing its job, not failing at it.

Where we stand today

What already exists from our earlier work, and what still has to be built.

What we already have

Between our two earlier projects we have a working front end and the plumbing behind it. In plain terms:

- **A working web interface.** A code editor in the browser, a panel that displays the AI response in readable formatted text, and a button to connect the two. This is genuinely reusable - the same editor component can show a side-by-side before-and-after comparison, which is exactly what we need.
- **A working connection to an AI model.** Code goes to a language model, a response comes back, and it is cleaned up and displayed. The model and the instructions will change, but the path is proven.
- **User accounts and request limits.** Sign-in, and a limit on how many times one user can run the tool per minute. Already written and working.
- **A safe way to run code, in outline.** One project already sends code away to be run somewhere isolated rather than on our own machine, with size limits, a time limit and clean handling of errors. The service it uses is too limited for us, but the surrounding design is right and we will keep it.
- **Deployment setup.** Configuration for putting the application online already exists.

What still has to be built

Everything that makes the tool understand a project rather than a file, and everything that decides whether a change is safe. None of this exists in either earlier project:

- **Reading a whole project.** Walking a repository, parsing every file properly, and building the map of what depends on what. Today both projects accept a single block of pasted text and have no concept of a project at all.
- **Working out what a change affects.** Given one function, finding every other place in the project that uses it, and every test that covers it.
- **Choosing what to show the AI.** A whole project does not fit in an AI prompt. We need to decide what to include in full, what to include as a summary, and what to leave out.
- **The safety checks.** The full set of checks described on the previous page. This is the heart of the project and none of it exists yet.
- **The retry loop.** Turning a failed check into a precise, machine-readable explanation that the AI can act on, and knowing when to stop trying.
- **Undo.** Applying a change in a way that can be perfectly reversed if a check fails.
- **Measurement.** A set of test projects and a way to compare our tool against a plain AI, so we can report numbers rather than opinions.

Plan and proof

How the work is staged

Deliberately, the part that reads and maps the project is built and working before the AI is connected at all. That way, if the AI turns out to be disappointing, the project still stands on its own.

WHEN	PHASE	WHAT GETS DONE
Weeks 1-4	**Understanding code**	Build the part that reads a repository and works out what depends on what. Demonstrate it by asking "what breaks if I change this function?" and checking the answer by hand.
Weeks 5-7	**Bringing in the AI**	Connect the AI model, decide what to show it, and get a rewritten piece of code back in a usable form. No safety checks yet.
Weeks 8-11	**The safety checks**	Build the checks and the retry loop. Demonstrate it by deliberately feeding in a change that breaks something and showing the tool catch it and name the affected files.
Weeks 12-14	**Interface and results**	Finish the screen, run the comparison experiments, and write up the numbers.

How we will show that it works

Not just a demo. Two things, both measurable:

- **A fair comparison.** Run the same set of improvement tasks four ways: a plain AI with no project awareness, then progressively with our map, our checks, and our retry loop. Count how often each one produces a change that quietly breaks the project.
- **A deliberate trap.** We write fifty changes by hand, each one looking perfectly reasonable but each containing exactly one hidden break. We feed them to our checks and count how many are caught. This measures our contribution directly, without depending on how well the AI happens to perform that day.

The demonstration we are aiming for

The AI proposes a genuinely nicer version of a function. Our tool refuses it, and prints the four other places in the project that would have stopped working.

That refusal is a stronger result than a demo that simply produces tidier code - it is the tool proving it understands the project, which is exactly what the project set out to show.